

Capitolo 01 - Fondamenti

Caratteristiche principali di Java

Java continua a essere uno dei linguaggi di programmazione più diffusi e utilizzati al mondo. Nato negli anni '90 da un team guidato da James Gosling e rilasciato da Sun Microsystems nel 1995, Java si è evoluto costantemente per rispondere alle esigenze dello sviluppo moderno, mantenendo al tempo stesso compatibilità, stabilità e affidabilità.

Oggi Java rappresenta una delle principali piattaforme per lo sviluppo di applicazioni enterprise, servizi cloud, sistemi distribuiti, applicazioni web, microservizi, strumenti di automazione e applicazioni desktop.

1. Orientato agli oggetti

Java adotta il paradigma della programmazione orientata agli oggetti (Object-Oriented Programming - OOP), favorendo la modularità, il riutilizzo del codice e la manutenibilità delle applicazioni.

I principi fondamentali sono:

- Incapsulamento
- Ereditarietà
- Polimorfismo
- Astrazione

Le evoluzioni più recenti del linguaggio hanno introdotto strumenti che semplificano la modellazione del dominio:

- Record
- Sealed Classes
- Pattern Matching
- Record Patterns

Queste funzionalità consentono di realizzare modelli più sicuri, leggibili e meno soggetti a errori.

2. Gestione automatica della memoria

Java offre una gestione automatica della memoria attraverso il Garbage Collector (GC).

Lo sviluppatore non deve occuparsi direttamente dell'allocazione e della deallocazione della memoria come avviene in linguaggi quali C o C++.

Le implementazioni moderne della JVM mettono a disposizione diversi algoritmi di garbage collection ottimizzati per differenti scenari:

- G1 Garbage Collector
- ZGC
- Shenandoah

Questi sistemi consentono di ridurre i tempi di pausa e migliorare le prestazioni delle applicazioni ad alta scalabilità.

3. Portabilità e compatibilità

Uno dei principi storici di Java è:

Write Once, Run Anywhere

Il codice Java viene compilato in bytecode ed eseguito dalla Java Virtual Machine (JVM), disponibile per numerosi sistemi operativi e architetture hardware.

I vantaggi principali sono:

- Portabilità tra piattaforme
- Compatibilità tra versioni
- Aggiornamenti continui della JVM
- Ampio ecosistema di librerie e framework

La forte attenzione alla retrocompatibilità rappresenta ancora oggi uno dei punti di forza della piattaforma Java.

4. Sicurezza e robustezza

La sicurezza è stata uno degli obiettivi fondamentali di Java fin dalla sua progettazione.

Tra i meccanismi principali troviamo:

- Verifica del bytecode
- Controllo rigoroso dei tipi
- Gestione delle eccezioni
- Controllo degli accessi
- API crittografiche avanzate

Le versioni moderne della piattaforma integrano inoltre continui aggiornamenti di sicurezza e strumenti per la gestione sicura delle dipendenze software.

5. Concorrenza e Parallelismo

Java supporta la programmazione concorrente fin dalle prime versioni attraverso:

- Thread
- Sincronizzazione
- Executor Framework
- Fork/Join Framework
- CompletableFuture

L'introduzione dei Virtual Threads ha rappresentato una delle innovazioni più significative degli ultimi anni.

I Virtual Threads consentono di gestire centinaia di migliaia di attività concorrenti con un impatto minimo sulle risorse del sistema, semplificando notevolmente lo sviluppo di applicazioni scalabili.

6. Programmazione Funzionale

Java integra funzionalità di programmazione funzionale che consentono di scrivere codice più espressivo e conciso.

Tra le principali:

- Lambda Expressions
- Method References
- Functional Interfaces
- Stream API
- Optional

Questi strumenti permettono di elaborare collezioni e flussi di dati in modo dichiarativo, migliorando leggibilità e produttività.

7. Ecosistema Enterprise

Java dispone di uno degli ecosistemi più maturi e completi nel panorama dello sviluppo software.

Framework e tecnologie largamente utilizzati includono:

- Spring Framework
- Spring Boot
- Jakarta EE
- Hibernate
- JPA
- Maven
- Gradle

Grazie a questi strumenti è possibile sviluppare applicazioni enterprise, microservizi, API REST e sistemi cloud-native.

8. Cloud e Microservizi

Java è una delle piattaforme più utilizzate nello sviluppo cloud.

Le principali caratteristiche comprendono:

- Supporto ai container Docker
- Integrazione con Kubernetes
- Architetture a microservizi
- Osservabilità e monitoring
- Scalabilità orizzontale

Framework come Spring Boot e Spring Cloud facilitano la realizzazione di applicazioni distribuite moderne.

Versioni di Java

Dal passaggio al ciclo di rilascio semestrale, Java evolve attraverso aggiornamenti regolari che introducono nuove funzionalità e miglioramenti della piattaforma.

Le organizzazioni che privilegiano stabilità e supporto a lungo termine adottano generalmente le versioni LTS (Long-Term Support).

Principali Versioni LTS

Versione	Rilascio
Java SE 8	Marzo 2014
Java SE 11	Settembre 2018
Java SE 17	Settembre 2021
Java SE 21	Settembre 2023
Java SE 25	Settembre 2025

Le versioni LTS rappresentano il riferimento per gli ambienti di produzione e ricevono aggiornamenti di manutenzione e sicurezza per un periodo prolungato.

Evoluzione recente del linguaggio

Tra le innovazioni introdotte nelle versioni più recenti troviamo:

- Records
- Pattern Matching
- Virtual Threads
- String Templates
- Sequenced Collections
- Scoped Values
- Structured Concurrency
- Miglioramenti alle API della JVM
- Ottimizzazioni del Garbage Collector
- Evoluzione delle API concorrenti

Queste funzionalità rendono Java più espressivo, moderno ed efficiente, mantenendo la compatibilità con il codice esistente.

Conclusioni

Java continua a rappresentare una delle piattaforme di sviluppo più solide e complete disponibili. La combinazione di stabilità, retrocompatibilità, sicurezza, prestazioni e innovazione costante lo rende una

scelta eccellente per lo sviluppo di applicazioni enterprise, servizi cloud, sistemi distribuiti e software professionale di ogni dimensione.

L'evoluzione continua del linguaggio, unita alla maturità dell'ecosistema e alla disponibilità di framework come Spring Boot, garantisce a Java un ruolo centrale nello sviluppo software moderno.

Espressioni

Le espressioni in programmazione sono combinazioni di valori, operatori e chiamate di funzioni che possono essere valutate per produrre un risultato. Le espressioni possono rappresentare calcoli aritmetici, valutazioni booleane, concatenazioni di stringhe e altro ancora. Le espressioni sono fondamentali per la creazione di logica e la manipolazione dei dati all'interno di un programma. Ecco alcuni esempi di espressioni:

Espressioni in Java (2024)

In Java, un'espressione è una combinazione di **operandi** (variabili, valori o oggetti) e **operatori** (simboli che eseguono operazioni su uno o più operandi). Un'espressione calcola un valore e può essere utilizzata in vari contesti, come assegnazioni, condizioni e loop.

Tipi di espressioni

1. Espressioni letterali

Un'espressione letterale è costituita da un valore costante (un numero, un carattere, ecc.).

- Esempi:

```
5          // Un intero
'A'        // Un carattere
3.14       // Un valore decimale (double)
true       // Un booleano
```

2. Espressioni di variabili

Un'espressione di variabile utilizza una variabile come operando.

- Esempio:

```
int x = 10;
x;    // Variabile che rappresenta il valore 10
```

3. Espressioni aritmetiche

Queste espressioni coinvolgono operazioni matematiche su numeri e variabili. Gli operatori principali sono:

- `+` (somma), `-` (sottrazione), `*` (moltiplicazione), `/` (divisione), `%` (modulo)

- Esempio:

```
int somma = 10 + 5;           // 15
int prodotto = 2 * 3;         // 6
int modulo = 10 % 3;          // 1 (resto)
```

4. Espressioni di assegnazione

Le espressioni di assegnazione memorizzano il risultato di un'espressione in una variabile.

- Operatore: =
- Esempio:

```
int x;
x = 5 + 3;    // Assegna 8 a x
```

5. Espressioni boolean

Espressioni che restituiscono `true` o `false`, spesso utilizzate nelle condizioni.

- Operatori logici:
 - `&&` (and), `||` (or), `!` (not)
- Operatori di confronto:
 - `==` (uguale), `!=` (diverso), `<`, `>`, `<=`, `>=`
- Esempio:

```
boolean isEqual = (5 == 5); // true
boolean isGreater = (10 > 3); // true
```

6. Espressioni condizionali (ternarie)

L'operatore ternario valuta un'espressione e restituisce un valore in base a una condizione.

- Sintassi:

```
condizione ? valoreSeTrue : valoreSeFalse;
```

- Esempio:

```
int max = (a > b) ? a : b;
```

Gli operatori in Java possono essere classificati in base alla loro funzione:

Operatori aritmetici

Utilizzati per eseguire operazioni matematiche.

- `+` (somma)
 - `-` (sottrazione)
 - `*` (moltiplicazione)
 - `/` (divisione)
 - `%` (modulo)
-

Operatori di assegnazione

Permettono di assegnare il valore di un'espressione a una variabile.

- `=` (assegnazione semplice)
- Operatori di assegnazione combinati:
 - `+=`, `-=`, `*=`, `/=`, `%=`
 - Esempio:

```
int a = 10;  
a += 5; // Equivale a: a = a + 5;
```

Operatori di confronto

Questi operatori restituiscono un valore booleano (`true` o `false`).

- `==` (uguale)
 - `!=` (diverso)
 - `<` (minore)
 - `>` (maggiore)
 - `<=` (minore o uguale)
 - `>=` (maggiore o uguale)
-

Operatori logici

Utilizzati nelle espressioni booleane per combinare più condizioni.

- `&&` (and logico)
 - `||` (or logico)
 - `!` (not logico)
-

Operatori di incremento/decremento

Usati per aumentare o diminuire il valore di una variabile di 1.

- `++` (incremento)
- `--` (decremento)

Pre-incremento e post-incremento:

- `++x` (incrementa prima di usare il valore)
- `x++` (incrementa dopo aver usato il valore)

Esempio:

```
int a = 5;
int b = ++a; // a viene incrementato prima, quindi b = 6
int c = a--; // a viene usato prima, quindi c = 6 e poi a = 5
```

Ordine di valutazione: precedenza e associatività degli operatori

In un'espressione complessa, l'ordine con cui gli operatori vengono valutati dipende dalla loro **precedenza** e **associatività**.

- **Precedenza:** gli operatori con precedenza più alta vengono valutati per primi.
 - Ad esempio: `*` e `/` hanno precedenza più alta di `+` e `-`.
- **Associatività:** quando due operatori hanno la stessa precedenza, l'**associatività** determina l'ordine di valutazione.
 - Ad esempio, la maggior parte degli operatori aritmetici sono **associativi da sinistra**:

```
int x = 5 - 3 + 2; // (5 - 3) + 2 = 4
```

- L'operatore di assegnazione `=` è **associativo da destra**:

```
int x, y;
x = y = 5; // Assegna 5 a y, poi a x
```

Espressioni con casting

In alcuni casi, è necessario forzare la conversione di un tipo di dato in un altro. Questo processo è chiamato **casting**.

Esempio:


```
int x = 10;
double y = (double) x / 3; // Il cast forza la divisione in double
```

Espressioni di stringhe

In Java, si possono utilizzare le espressioni per **concatenare** stringhe utilizzando l'operatore `+`.

```
String nome = "Mario";
String saluto = "Ciao, " + nome; // Concatenazione di stringhe
```

Conclusione

Le espressioni in Java sono fondamentali per eseguire operazioni su dati, effettuare confronti, assegnare valori e controllare il flusso di un programma. La comprensione dei vari tipi di espressioni e degli operatori utilizzati in Java è essenziale per scrivere codice efficiente e corretto.

[esempi](#)

Operatori nei linguaggi di programmazione

Negli linguaggi di programmazione, gli operatori sono simboli speciali o parole chiave che eseguono operazioni su uno o più operandi. Gli operandi sono i valori o le variabili su cui l'operatore agisce. Gli operatori sono fondamentali per eseguire operazioni aritmetiche, logiche, di confronto e altre azioni specifiche all'interno di un programma. Ecco una breve definizione di alcuni tipi comuni di operatori:

1. Operatori Aritmetici:

- Eseguono operazioni matematiche come l'addizione, la sottrazione, la moltiplicazione e la divisione.
- Esempi: `+` (addizione), `-` (sottrazione), `*` (moltiplicazione), `/` (divisione).

2. Operatori di Confronto o Relazionali:

- Confrontano due valori e restituiscono un valore booleano che indica se la relazione è vera o falsa.
- Esempi: `==` (uguale a), `!=` (diverso da), `<` (minore di), `>` (maggiore di), `<=` (minore o uguale a), `>=` (maggiore o uguale a).

3. Operatori Logici:

- Eseguono operazioni logiche su valori booleani. Solitamente utilizzati in strutture di controllo decisionale.
 - Esempi: `&&` (AND logico), `||` (OR logico), `!` (NOT logico).
-

4. Operatori di Assegnamento:

- Assegnano un valore a una variabile.
- Esempio: `=` (assegnamento), `+=` (assegnamento con somma), `-=` (assegnamento con sottrazione), `*=` (assegnamento con moltiplicazione), `/=` (assegnamento con divisione).

5. Operatori di Incremento e Decremento:

- Modificano il valore di una variabile incrementandolo o decrementandolo di una certa quantità.
- Esempi: `++` (incremento), `--` (decremento).

6. Operatori Bitwise:

- Eseguono operazioni bit a bit su numeri interi.
- Esempi: `&` (AND bit a bit), `|` (OR bit a bit), `^` (XOR bit a bit), `~` (NOT bit a bit), `<<` (shift a sinistra), `>>` (shift a destra).

7. Operatori Ternari:

- Sono operatori condizionali che valutano una condizione e restituiscono un valore in base al risultato della condizione.
- Esempio: `condizione ? valore_se_vero : valore_se_falso.`

Gli operatori sono essenziali per manipolare dati e controllare il flusso di esecuzione all'interno di un programma, consentendo la creazione di logica complessa e la gestione di variabili e valori.

Operatori in Java

Operatori aritmetici

- Di assegnazione: `=` `+=` `-=` `*=` `/=` `&=` `|=` `^=`
- Di assegnazione/incremento: `++` `--` `%=`
- Operatori Aritmetici: `+` `-` `*` `/` `%`

Operatore	Significato
<code>+</code>	addizione
<code>-</code>	sottrazione
<code>*</code>	moltiplicazione
<code>/</code>	divisione
<code>%</code>	resto
<code>++var</code>	preincremento
<code>--var</code>	predecremento
<code>var++</code>	postincremento

Operatore	Significato
<code>var--</code>	postdecremento

Operatori di assegnazione

Operatore	Significato
<code>=</code>	assegnazione
<code>+=</code>	addizione assegnazione
<code>-=</code>	sottrazione assegnazione
<code>*=</code>	motiplicazione assegnazione
<code>/=</code>	divisione assegnazione
<code>%=</code>	resto assegnazione

Operatori relazionali

`==` `!=` `>` `<` `>=` `<=`

Operatore	Significato
<code><</code>	minore di
<code><=</code>	minore di o uguale a
<code>></code>	maggiore di
<code>>=</code>	maggiore di o uguale a
<code>==</code>	uguale a
<code>!=</code>	non uguale / diverso

Operatori per Booleani

- Bitwise (interi): `&` `|` `^` `<<` `>>` `~`

Operatore	Significato
<code>&&</code>	AND logico
<code>,</code>	
<code>!</code>	NOT
<code>^</code>	exclusive OR

Attenzione:

- Gli operatori logici agiscono **solo su booleani**
 - Un intero NON viene considerato un booleano
 - Gli operatori relazionali forniscono valori booleani
-

Operatori su reference

Per i riferimenti/reference, sono definiti:

- Gli operatori relazionali == e !=
 - test sul riferimento all'oggetto, **NON sull'oggetto**
 - Le assegnazioni
 - L'operatore "punto"
 - NON è prevista l'aritmetica dei puntatori, vengono gestiti dalla JVM
-

Operazioni matematiche complesse

Operazioni matematiche complesse sono permesse dalla **classe Math** (package java.lang)

- `Math.sin (x)` calcola $\sin(x)$
- `Math.sqrt (x)` calcola $x^{(1/2)}$
- `Math.PI` ritorna pi
- `Math.abs (x)` calcola $|x|$
- `Math.exp (x)` calcola e^x
- `Math.pow (x, y)` calcola x^y

Esempio

- `z = Math.sin (x) - Math.PI / Math.sqrt(y)`
-

Comparazione del tipo di dato: Type Comparison Operator

- `instanceof` - Verifica se un certo oggetto è istanza di un certo Tipo di dato
 - p.es: `if (a instanceof Automobile) //fai qualcosa`
-

Caratteri speciali

Literal	Represents
<code>\n</code>	New line
<code>\t</code>	Horizontal tab
<code>\b</code>	Backspace
<code>\r</code>	Carriage return
<code>\f</code>	Form feed

Literal	Represents
<code>\\</code>	Backslash
<code>\"</code>	Double quote
<code>\ddd</code>	Octal character
<code>\xdd</code>	Hexadecimal character
<code>\udddd</code>	Unicode character

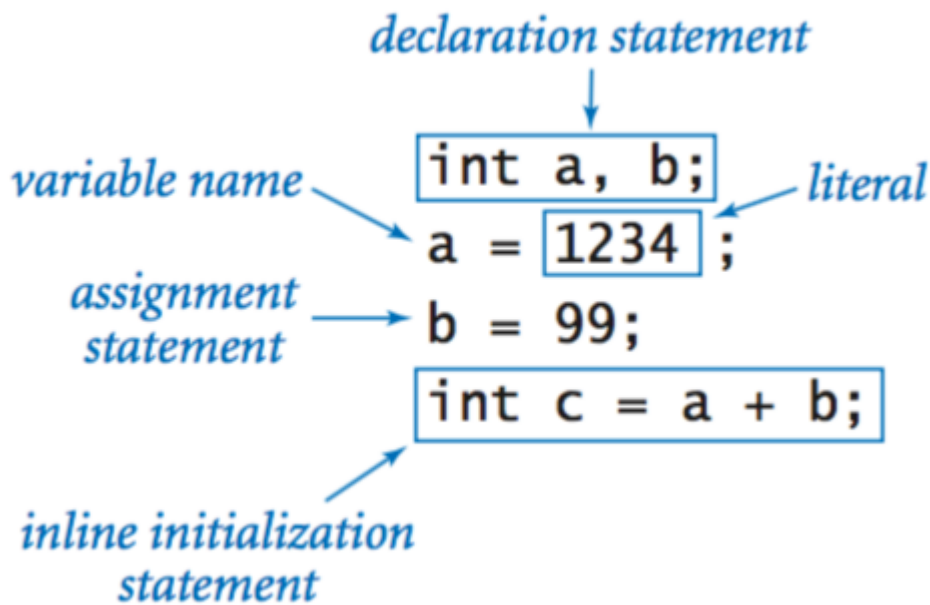
- [raccolta esempi](#)
- [altri esempi](#)

Le variabili e le costanti in Java (2024)

In Java, **variabili** e **costanti** sono i fondamenti per la gestione dei dati. Le variabili rappresentano delle aree di memoria destinate a memorizzare informazioni che possono cambiare durante l'esecuzione del programma, mentre le costanti sono utilizzate per memorizzare valori che rimangono invariati. Con le versioni avanzate del linguaggio Java, il concetto di variabili e costanti è rimasto centrale per il funzionamento e l'efficienza dei programmi.

Variabili: concetti chiave

- Una **variabile** è un'area di memoria identificata da un **nome**.
 - Il suo scopo è quello di contenere un valore di un **certo tipo**.
 - Le variabili sono usate per **memorizzare dati** durante l'esecuzione del programma.
 - Il nome di una variabile è un **identificatore**:
 - Può contenere lettere, numeri e l'underscore `_`.
 - Non deve coincidere con una parola chiave del linguaggio Java (ad es., `int`, `class`).
 - Si consiglia di scegliere un **nome significativo** per migliorare la leggibilità del programma.
-



Esempio di utilizzo:

```
public class Triangolo {  
    public static void main(String[] args) {  
        int base, altezza; // Dichiarazione di variabili locali  
        double area;  
  
        base = 5;  
        altezza = 10;  
        area = base * altezza / 2;  
  
        System.out.println(area);  
    }  
}
```

Questo esempio mostra come l'uso delle variabili renda il programma più **chiaro** e più facile da comprendere.

Dichiarazione di variabili

- In Java, **ogni variabile deve essere dichiarata** prima del suo utilizzo.
- La dichiarazione richiede la specifica del **nome** della variabile e del suo **tipo**.

Esempio:

```
int base, altezza;  
double area;
```

Regole per le variabili:

- Ogni variabile deve essere dichiarata **una sola volta**.
 - Le variabili non possono essere utilizzate finché non sono state **inizializzate**.
-

Assegnazione:

L'assegnazione permette di **memorizzare un valore** in una variabile. Un'assegnazione può essere effettuata con un letterale o con il risultato di un'espressione.

Esempio:

```
base = 5;
altezza = 10;
area = base * altezza / 2;
```

Dichiarazione + Assegnazione:

Si può combinare la dichiarazione e l'assegnazione in un'unica linea:

```
int base = 5;
int altezza = 10;
double area = base * altezza / 2;
```

Scope: ambito di visibilità delle variabili

Lo **scope** di una variabile definisce dove essa può essere utilizzata nel programma. Le variabili possono essere **locali** (definite all'interno di un metodo o di un blocco) o **globali** (definite a livello di classe).

Esempio:

```
class Nascoste {
    static int x, y; // Variabili globali

    static void f() {
        int x;
        x = 1; // Variabile locale
        y = 1; // Variabile globale
        System.out.println(x); // Stampa la variabile locale
        System.out.println(y); // Stampa la variabile globale
    }

    public static void main(String[] args) {
```

```
x = 0; // Variabile globale
y = 0; // Variabile globale
f();
System.out.println(x); // Stampa la variabile globale
System.out.println(y); // Stampa la variabile globale
}
}
```

Costanti

Le **costanti** sono variabili il cui valore **non può cambiare** dopo essere stato inizializzato. Per dichiarare una costante, si utilizza il modificatore `final`:

```
final double IVA = 0.22;
```

- Il modificatore **final** impedisce di assegnare un nuovo valore alla variabile dopo la sua inizializzazione.
- L'uso delle costanti può aiutare a **prevenire errori** e consente al compilatore di eseguire **ottimizzazioni**.

L'attributo `final`:

- **Variabile:** la trasforma in una costante.
- **Metodo:** impedisce l'overriding (ridefinizione) in una classe derivata.
- **Classe:** impedisce la derivazione di altre classi (la classe diventa una "foglia" nell'albero di ereditarietà).

Esempio di costanti:

```
final double PI = 3.14159;
final int MAX_VALUE = 100;
```

Input dell'utente

Per ottenere valori dall'utente, in Java si può utilizzare la classe `Scanner`, che appartiene al package `java.util`:

```
import java.util.Scanner;

public class InputExample {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
```



```
System.out.print("Inserisci un numero: ");
int numero = input.nextInt();
System.out.println("Hai inserito: " + numero);
}
}
```

Conclusione:

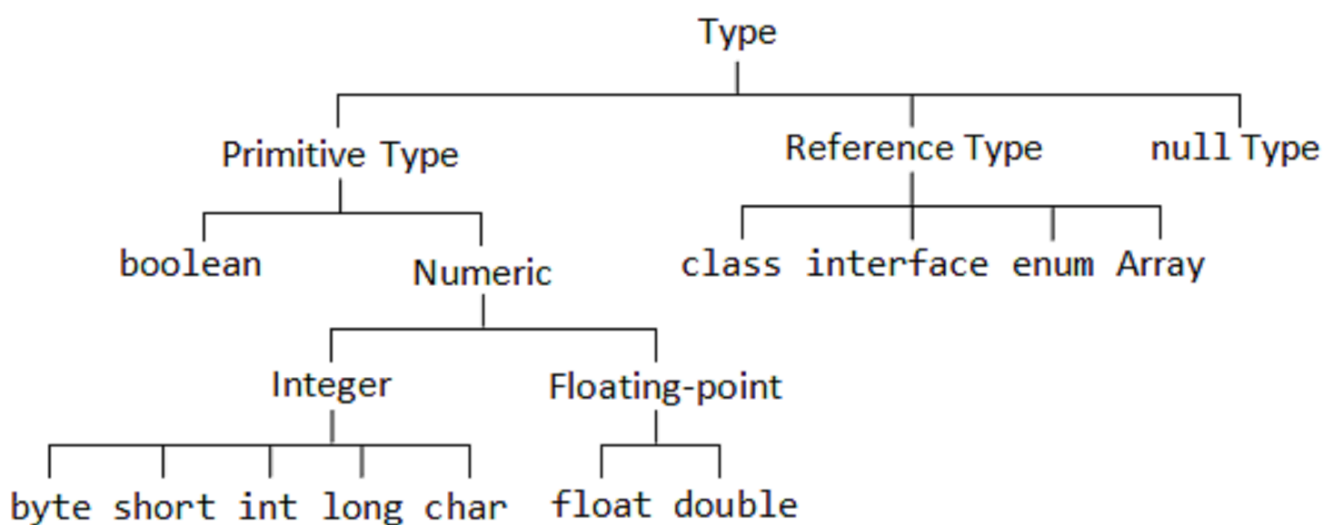
Le variabili e le costanti rappresentano due elementi fondamentali nella programmazione in Java. Le prime consentono di **memorizzare dati variabili** nel corso dell'esecuzione di un programma, mentre le seconde garantiscono la sicurezza e la stabilità delle informazioni che non devono cambiare. Utilizzare questi elementi in modo corretto aiuta a scrivere codice più chiaro, efficiente e meno incline agli errori.

Esempi e risorse aggiuntive:

- [Raccolta di esempi su variabili e costanti](#)
 - [Altri esempi](#)
-

Tipi di dato primitivi in Java (2024)

I **tipi di dato primitivi** rappresentano i costituenti fondamentali in Java. Essi forniscono le basi per la dichiarazione delle variabili e la gestione dei dati semplici, come numeri, caratteri e valori booleani. A differenza dei tipi di dato complessi, i tipi primitivi **non sono oggetti** e non possiedono metodi. Con l'evoluzione di Java fino al 2024, il loro ruolo rimane cruciale, specialmente per le operazioni di basso livello che richiedono efficienza e semplicità.



1. byte:

- **Dimensione:** 8 bit
- **Intervallo:** -128 a 127

- Utilizzato principalmente per risparmiare memoria in grandi array, soprattutto in file binari.
-

2. short:

- **Dimensione:** 16 bit
 - **Intervallo:** -32,768 a 32,767
 - Spesso utilizzato quando lo spazio in memoria è limitato, come per rappresentare numeri in applicazioni embedded.
-

3. int:

- **Dimensione:** 32 bit
 - **Intervallo:** -2,147,483,648 a 2,147,483,647
 - Il tipo più comunemente utilizzato per rappresentare numeri interi.
-

4. long:

- **Dimensione:** 64 bit
 - **Intervallo:** -9,223,372,036,854,775,808 a 9,223,372,036,854,775,807
 - Usato per numeri interi che superano l'intervallo dell'`int`.
-

5. float:

- **Dimensione:** 32 bit
 - **Precisione:** Circa 7 cifre decimali
 - Utilizzato per i numeri decimali quando la precisione richiesta non è molto elevata (come per i calcoli grafici o scientifici).
-

6. double:

- **Dimensione:** 64 bit
 - **Precisione:** Circa 15 cifre decimali
 - Il tipo preferito per rappresentare numeri decimali in applicazioni di calcolo più complesse.
-

7. char:

- **Dimensione:** 16 bit
 - **Intervallo:** 0 a 65,535 (rappresenta un singolo carattere Unicode)
 - Utilizzato per memorizzare caratteri singoli, tra cui simboli speciali e lettere in lingue diverse.
-

8. boolean:

- **Dimensione:** non specificata (generalmente 1 bit)

- Può assumere solo i valori `true` o `false`
 - Utilizzato principalmente per controllare il flusso dei programmi tramite condizioni logiche.
-

Evoluzione e ottimizzazioni (2024)

Nonostante Java sia giunto a versioni molto avanzate, i tipi primitivi restano cruciali. Le versioni più recenti del linguaggio, come Java 21, non hanno modificato la struttura di base di questi tipi, ma hanno ottimizzato ulteriormente la gestione della memoria e l'efficienza computazionale, rendendo i tipi primitivi ancora più performanti.

Esempi pratici di utilizzo

Dichiarazione di variabili primitive e stampa dei loro valori:

```
// Tipi interi
byte mioByte = 127;
System.out.println(mioByte);

short mioShort = 851;
System.out.println(mioShort);

long mioLong = 34093L;
System.out.println(mioLong);

// Tipi reali
float mioFloat = 324.4f;
System.out.println(mioFloat);

double mioDouble = 3.14159732;
System.out.println(mioDouble);

// Carattere e booleano
char mioChar = 'y';
System.out.println(mioChar);

boolean mioBoolean = true;
System.out.println(mioBoolean);
```

Dichiarazione e utilizzo di variabili locali:

```
// 1) Dichiarazione
int mioNumero;

// 2) Inizializzazione
mioNumero = 100;
```

```
// 3) Utilizzo della variabile locale
System.out.println(mioNumero);
```

Nota: Le variabili locali devono sempre essere inizializzate prima dell'utilizzo.

Tabelle riassuntive dei tipi di dato primitivi

Tipo	Dimensione
byte	8 bit
short	16 bit
int	32 bit
long	64 bit
float	32 bit
double	64 bit
char	16 bit
boolean	true/false

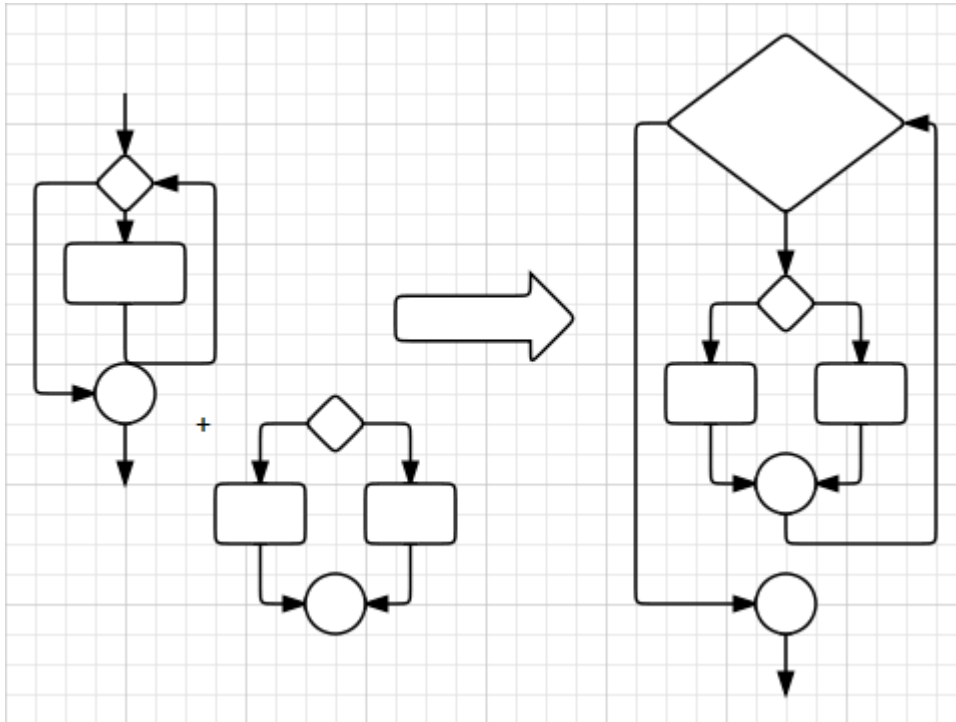
Conclusioni

I **tipi primitivi** in Java rimangono fondamentali per la gestione di dati semplici e per garantire l'efficienza nelle applicazioni. Con l'evoluzione del linguaggio, Java ha mantenuto il focus sulla robustezza e sull'efficienza di questi tipi, continuando a renderli la scelta preferita per le operazioni di base, specialmente in ambiti ad alte prestazioni come il machine learning, l'intelligenza artificiale e i sistemi embedded.

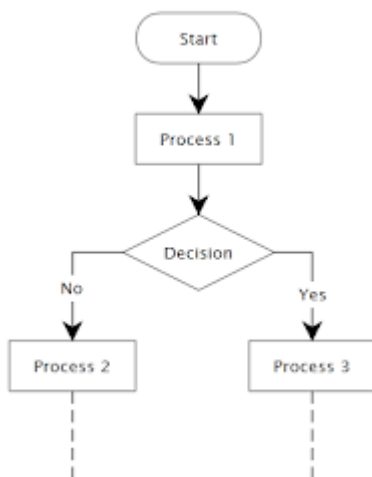
Il controllo del flusso

Java mette a disposizione del programmatore diverse strutture sintattiche per consentire il **controllo del flusso**

- IF – ELSE
- WHILE e DO-WHILE
- FOR e FOREACH
- SWITCH - CASE



Selezione, scelta condizionale



if statements

- E' un'istruzione condizionale, permette cioè di eseguire un blocco di istruzioni solo se si verifica una determinata condizione.

```

if (condition) {

    //statements;

}
  
```

else if [opzionale]

- Indichiamo anche cosa fare se non si supera la condizione

```
else if (condition2) {  
  
    //statements;  
  
}
```

else [opzionale]

```
else {  
  
    //statements;  
  
}
```

Switch Statements

Serve per gestire in maniera più ordinata varie condizioni, è un modo più elegante in alcune situazioni.

```
switch (espressione) {  
  
    case valore1:  
  
        //statements;  
  
        break;  
  
    ...  
  
    case valoren:  
  
        //statements;  
  
        break;  
  
    default:  
  
        //statements;  
  
}
```

```
}
```

Cicli definiti - for

Se il numero di iterazioni è **prevedibile**.

```
for (init; condition; adjustment) {  
  
    //statements;  
  
}
```

Esempio: prima di entrare nel ciclo so già che verrà ripetuto 10 volte

```
int n=10;  
for (int i=0; i<n; ++i) {  
    ...  
}
```

Cicli indefiniti - while

si ripete il ciclo fintanto che una condizione è verificata (è vera)

- la condizione booleana `true/false`
- determina la continuazione del programma
- ed esegue l'elenco delle operazioni del blocco
- Da usare se il numero di iterazioni **non è noto** all'inizio del ciclo.

```
while (condition) {  
    //statements;  
}  
  
//Esempio: quando il numero di iterazioni dipende da valori in input  
while(true) {  
    x = Integer.parseInt(JOptionPane.showInputDialog("Immetti numero  
positivo"));  
    if (x > 0) break;  
}
```

do-while

- Quando voglio eseguire almeno una volta l'istruzione, anche se la condizione è impostata su `false`
- Si verifica la condizione dopo il primo ciclo

```
do {  
  
    //statements;  
  
} while (condition);
```

Cicli con interruzione: break

Se il ciclo viene interrotto dopo aver filtrato un valore con una data proprietà.

- Se stiamo eseguendo un ciclo, possiamo utilizzare la parola `break` per interromperlo in qualsiasi momento.
- Si interrompe quindi il ciclo e si riprende l'esecuzione delle istruzioni al di fuori di esso.

Esempio: verifica se un array contiene o meno numeri negativi

```
boolean trovato = false;  
for (int i=0; i<v.length; ++i) // passa in rassegna tutti gli indici  
dell'array v  
    if (v[i]<0) { // filtra le celle che contengono valori negativi  
        trovato = true;  
        break; // interrompe ciclo  
    }  
// qui trovato vale true se e solo se vi sono numeri negativi in v
```

Cicli con salto: continue

Invece terminare completamente il ciclo ed uscire fuori, `continue` interrompe solo l'iterazione corrente e passa alla successiva.

```
{  
    int a;  
  
    for ( a = 1; a <= 10 ; a++ )           // Run a = 1, 2, ..., 10  
    {
```



```
        if ( a == 4 )
        {
            continue;           // Skip printing 4...
        }

        System.out.println(a);    // Print a
    }
}
```

Cicli con condizione

Vengono passati in rassegna un insieme di valori e per ognuno di essi viene fatto un test per verificare se il valore ha o meno una certa proprietà in base alla quale decideremo se prenderlo in considerazione o meno.

Esempio: stampa tutti i numeri pari fino a 100

```
for (int i=1; i<100; ++i) { // passa in rassegna tutti i numeri fra 1 e 100
    if (i % 2 == 0) // filtra quelli pari
        System.out.println(i);
}
```

Cicli con accumulatore

Vengono passati in rassegna un insieme di valori e ne viene tenuta una traccia cumulativa usando una opportuna variabile.

Esempio: somma i primi 100 numeri interi.

```
int somma = 0; // variabile accumulatore di tipo int
for (int i=1; i<100; ++i) { // passa in rassegna tutti i numeri fra 1 e 100
    somma = somma + i; // accumula i valori nella variabile accumulatore
}
```

Esempio: data una stringa s, ottieni la stringa rovesciata

```
String rovesciata = ""; // variabile accumulatore di tipo String
for (int i=0; i<s.length(); ++i) { // passa in rassegna tutti gli indici
    dei caratteri di s
        rovesciata = s.substring(i, i+1) + rovesciata; // accumula i caratteri
    in testa all'accumulatore
}
```

```
}
```

Cicli misti

Esempio di ciclo definito con filtro e accumulatore: calcola la somma dei soli valori positivi di un array

```
int somma = 0;
for (int i=0; i<v.length; ++i) // passa in rassegna tutti gli indici
    dell'array v
    if (v[i]>0) // filtra le celle che contengono valori positivi
        somma = somma + v[i]; // accumula valore nella variabile
    accumulatore
```

Cicli annidati

Se un ciclo appare nel corpo di un altro ciclo.

Esempio: stampa quadrato di asterischi di lato n

```
for (int i=0; i<n; i++) {
    for (int j=0; j<n; j++) System.out.print("*");
    System.out.println();
}
```

codice esempi d'uso

- [raccolta esempi](#)
- [altri esempi](#)
- [01_if-elseif-else](#)
- [02_switch](#)
- [03_while](#)
- [04_for](#)
- [05_foreach](#)
- [06_labels](#)
- [giochi](#)
- [programmi](#)

Array

- Sequenze ordinate di

- Tipi primitivi (int, float, etc.)
 - Riferimenti ad oggetti (vedere classi!)
 - Elementi dello stesso tipo
 - Raggruppati sotto lo stesso nome
 - Indirizzati da indici
 - Raggiungibili con l'operatore di indicizzazione
 - le **parentesi quadre** `[]`
-

Il riferimento ad array

- Non è un puntatore al primo elemento
 - È un puntatore all'oggetto array
 - Incrementandolo non si ottiene il secondo elemento
-

Cosa sono gli array?

Nel linguaggio di programmazione Java, gli array sono oggetti (§4.3.1), vengono creati dinamicamente e possono essere assegnati a variabili di tipo Object (§4.3.2). Tutti i metodi della classe Object possono essere invocati su un array. [Java docs](#)

In Java gli array sono Oggetti

- Sono allocati nell'area di memoria riservata agli oggetti creati dinamicamente (heap)
-

Dimensione dell'array

- Può essere stabilita a run-time (quando l'oggetto viene creato)
 - È **fissa** (non può essere modificata)
 - E' **nota** e ricavabile per ogni array
-

Creazione di un Array

L'operatore new crea un array:

- Con costante numerica

```
int[] voti;  
...  
voti = new int[10];
```

- Con costante simbolica

```
final int ARRAY_SIZE = 10;  
int[] voti;
```

```
...  
voti = new int[ARRAY_SIZE];
```

- Con valore definito a run-time

```
int[] voti;  
... definizione di x (run-time) ...  
voti = new int[x];
```

riempire l'array utilizzando un inizializzatore

- L'operatore new inizializza le variabili
 - 0 - per variabili di tipo numerico (inclusi i char)
 - false - per le variabili di tipo boolean

```
int[] numeriPrimi = {2,3,5,7,11,13};  
//...  
// La virgola finale e' facoltativa  
int [] numeriPari = {0, 2, 4, 6, 8, 10,};
```

-
- Dichiarazione e creazione possono avvenire contestualmente
 - L'attributo length indica la lunghezza dell'array, cioè il numero di elementi
 - Gli elementi vanno da 0 a length-1

```
for (int i=0; i<voti.length; i++)  
voti[i] = i;
```

In Java viene fatto il bounds checking

- Maggior sicurezza
- Accesso più lento

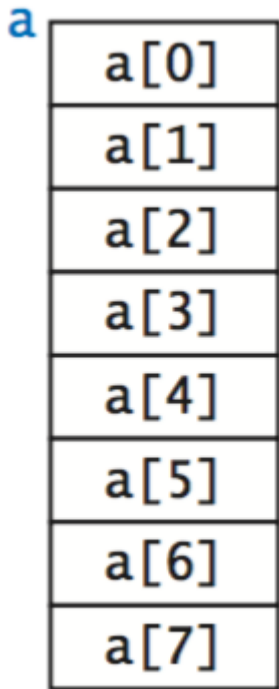
Array Mono-dimensionali (vettori)

Dichiarazione di un riferimento a un array

- `int[] voti;`
 - `int voti[];`
-

La dichiarazione di un array non assegna alcuno spazio

```
voti == null
```



Array Multi-dimensionali

- **Array contenenti riferimenti ad altri array**
- Sintatticamente sono estensioni degli array a una dimensione
- matrice: array di array, regolare, irregolare, rettangolare...

	99	85	98
row 1 →	98	57	78
	92	77	76
	94	32	11
	99	34	22
	90	46	54
	76	59	88
	92	66	89
	97	71	24
	89	29	38

column 2

- Le righe non sono memorizzate in posizioni adiacenti
- Possono essere spostate facilmente

```
// Scambio di due righe
double[][] saldo = new double[5][6];
...
double[] temp = saldo[i];
saldo[i] = saldo[j];
saldo[j] = temp;
```

- L'array è una struttura dati efficiente ogni volta che il numero di elementi è noto
- Il ridimensionamento di un array in Java risulta poco efficiente
- Utilizzare altre strutture dati se il numero di elementi contenuto non è noto

Esempi di Array

Array Monodimensionali

```
int[] list = new int[10];
list.length;
```

```
int[] list = {1, 2, 3, 4};
```

Array Multidimensionali

```
int[][] list = new int[10][10];  
list.length;  
list[0].length;  
int[][] list = {{1, 2}, {3, 4}};
```

Array irregolari Sono possibili righe di lunghezza diverse**

```
int[][] m = {  
    {1, 2, 3, 4},  
    {1, 2, 3},  
    {1, 2},  
    {1}  
};
```

Array di oggetti

Per gli array di oggetti (e.g., Integer) `Integer [] voti = new Integer [5];` ogni elemento e' un riferimento

L'inizializzazione va completata con quella dei singoli elementi

```
voti[0] = new Integer (1);  
voti[1] = new Integer (2);  
...  
voti[4] = new Integer (5);
```

Algoritmi per lavorare con gli array: la classe `java.util.Arrays`

- Il pacchetto `java.util` contiene metodi statici di utilità per gli array
- Copia di un valore in tutti gli (o alcuni) elementi di un array

- `Arrays.fill (<array>, <value>);`
- `Arrays.fill (<array>, <from>, <to>, <value>);`

- Copia di array

- `System.arraycopy (<arraySrc>, <offsetSrc>,<arrayDst>, <offsetDst>,<#elements>);`

- Confronta due array

- `Arrays.equals (<array1>, <array2>);`

- Ordina un array (di oggetti che implementino l'interfaccia Comparable)

- `Arrays.sort (<array>);`

- Ricerca binaria (o dicotomica)

- `Arrays.binarySearch (<array>);`

altre risorse

- [raccolta esempi](#)
- [altri esempi](#)
- [esempi ed esercizi su array](#)

Stringhe e Caratteri in Java

La gestione delle stringhe è fondamentale in Java. La classe `String` e le relative utility offrono un'ampia varietà di metodi e funzionalità per manipolarle.

♦ Creazione di Stringhe

```
// Dichiarazione e inizializzazione
String str1 = "Hello, World!";

// Con costruttore (sconsigliato salvo casi particolari)
String str2 = new String("Java String");

// Concatenazione
String concat = str1 + " " + str2;
```

♦ Metodi Principali

```
String s = "Java Programming";

// Lunghezza
int len = s.length();           // 16
```



```
// Carattere in posizione
char c = s.charAt(5);           // 'P'

// Confronto
boolean eq = s.equals("Java");  // false
boolean eqIgnore = s.equalsIgnoreCase("java programming"); // true

// Contiene sottostringa
boolean found = s.contains("gram"); // true

// Sottostringhe
String sub = s.substring(0, 4); // "Java"

// Maiuscole / minuscole
String up = s.toUpperCase();
String low = s.toLowerCase();

// Trim
String t = "  test  ".trim();   // "test"
```

◆ Concatenazione di Stringhe

```
// Operatore +
String fullName = firstName + " " + lastName;

// Metodo concat()
String greet = "Hello".concat(" World");

// StringBuilder (efficiente in loop)
StringBuilder sb = new StringBuilder();
sb.append("Java").append(" Rocks!");
String result = sb.toString();
```

◆ Formattazione

```
String name = "Alice";
int age = 30;

// String.format
String msg = String.format("Ciao %s, hai %d anni.", name, age);

// printf
System.out.printf("Ciao %s, hai %d anni.%n", name, age);

// MessageFormat (utile per i18n)
```

```
import java.text.MessageFormat;
String text = MessageFormat.format("Utente {0}, punti {1}", name, age);
```

◆ Conversione tra Stringhe e Tipi Primitivi

```
// String → numeri
int i = Integer.parseInt("123");
double d = Double.parseDouble("3.14");

// Numeri → String
String s1 = String.valueOf(456);
String s2 = Double.toString(3.14);

// String → boolean
boolean b = Boolean.parseBoolean("true");
```

◆ Manipolazione di Caratteri

```
String s = "Hello World";

// Indice di un carattere
int idx = s.indexOf('W'); // 6
int last = s.lastIndexOf('o'); // 7

// Sostituire caratteri o sottostringhe
String r1 = s.replace('l', 'x'); // Hexxo Worxd
String r2 = s.replace("World", "Java"); // Hello Java

// Split
String[] parts = s.split(" "); // ["Hello", "World"]

// Array di caratteri
char[] chars = s.toCharArray();
```

◆ Null e Stringhe Vuote

```
if (s == null || s.isEmpty()) {
    // nulla o stringa vuota
}

if (s == null || s.trim().isEmpty()) {
    // nulla o solo spazi
}
```

◆ Classi disponibili

- `String` → **immutabile**, final.
- `StringBuilder` → mutabile, non thread-safe (più usata).
- `StringBuffer` → mutabile, thread-safe (quasi mai usata oggi).
- `Character` → wrapper di `char`.

◆ Text Blocks (Java 13+)

Permettono di scrivere stringhe multilinea leggibili:

```
String json = """
    {
        "nome": "Alice",
        "eta": 30
    }
    """;
```

◆ String Templates (Java 21+, preview)

Nuova interpolazione di stringhe:

```
String nome = "Luca";
int punti = 120;

String msg = STR."Benvenuto \{nome}, hai \{punti} punti.";
System.out.println(msg);
// Output: Benvenuto Luca, hai 120 punti.
```

👉 Supportano anche espressioni:

```
int a = 5, b = 3;
String res = STR."Somma: \{a+b}"; // Somma: 8
```

✓ Conclusione

- `String` rimane immutabile → ogni operazione crea una nuova istanza.
 - Per modifiche ripetute, usare `StringBuilder`.
 - Per testo leggibile su più righe, usare **Text Blocks**.
 - Per interpolazione diretta, da Java 21 sono disponibili le **String Templates** (in preview).
-

Domande

Qual è la caratteristica principale della classe `String` in Java? A. È mutabile B. È immutabile C. È thread-safe D. È una sottoclasse di `StringBuilder`

Quale metodo restituisce la lunghezza di una stringa? A. `size()` B. `count()` C. `length()` D. `len()`

Cosa restituisce l'istruzione `"Java".charAt(2)`? A. 'J' B. 'a' C. 'v' D. 'A'

Quale tra i seguenti è più efficiente per concatenazioni ripetute in un ciclo? A. `String` B. `StringBuffer` C. `StringBuilder` D. `Character`

Quale metodo si usa per verificare se una stringa è vuota? A. `isNull()` B. `isEmpty()` C. `isBlank()` D. `equals("")`

Come si converte il numero intero 123 in una stringa? A. `Integer.toString(123)` B. `String.valueOf(123)` C. `"" + 123` D. Tutte le precedenti

Cosa produce il seguente codice? `String s = " Hello "; System.out.println(s.trim());` A. " Hello " B. "Hello" C. " Hello" D. "Hello "

Quale classe usare per gestire stringhe mutabili in modo NON thread-safe? A. `String` B. `StringBuilder` C. `StringBuffer` D. `Character`

Qual è il vantaggio dei Text Blocks introdotti in Java 13+? A. Permettono di creare stringhe multilinea leggibili B. Sono più veloci delle stringhe normali C. Rendono le stringhe immutabili D. Consentono la concatenazione automatica

Quale istruzione con String Templates (Java 21+) stampa "Ciao Luca" se `nome="Luca"`? A. `STR."Ciao {nome}"` B. `STR."Ciao {nome}"` C. `String.format("Ciao %s", nome)` D. `STR."Ciao nome"`

Casting e Promotion in Java

Il **casting** e la **promotion** sono due meccanismi che consentono di convertire valori da un tipo primitivo o reference a un altro.

◆ Casting dei tipi primitivi

- `(tipo) variabile`
- `(tipo) espressione`

👉 Trasforma il valore della variabile (o espressione) in un altro tipo.

Regole generali

- Il **cast esplicito** è richiesto quando si passa da un tipo più "grande" a uno più "piccolo".
- La **promotion** avviene automaticamente quando serve (es. `int` → `long`, `int` → `double`).
- Il cast si applica anche a `char` (che internamente è un intero positivo).
- Il cast può causare **perdita di informazione**.

Esempio: cast e differenza di ordine di valutazione

```
int i = (int) 3.0 * (int) 4.5;    // i = 12
int j = (int) (3.0 * 4.5);      // j = 13
```

👉 Nel primo caso ogni valore viene troncato prima della moltiplicazione, nel secondo dopo.

◆ Promotion (conversione implicita)

La **promotion** è automatica quando i tipi sono compatibili e non si rischia perdita di dati.

```
char a = 'a';
int b = a;    // char → int (promotion automatica)

System.out.println(a); // a
System.out.println(b); // 97
```

Altro esempio:

```
int valore = 56;
long valoreLong = valore; // int → long
System.out.println(valoreLong); // 56
```

◆ Type Casting (esplicito)

Quando si forza un valore in un tipo più piccolo o incompatibile, si parla di **cast esplicito**.

```
byte b = (byte) 261;
System.out.println(b); // 5

System.out.println(Integer.toBinaryString(b)); // 101 (valore troncato)
System.out.println(Integer.toBinaryString(261)); // 100000101
```

```
double d = 1936.27;
int a = (int) d; // troncamento
System.out.println(a); // 1936
```

♦ Casting e boolean

⚠ Con il tipo `boolean` **non è possibile** fare cast verso o da tipi numerici:

```
int a = (int) true; // ERRORE: boolean cannot be converted to int
boolean b = (boolean) 0; // ERRORE: int cannot be converted to boolean
```

♦ Casting dei tipi reference (oggetti)

Il cast sui tipi reference è consentito **solo con ereditarietà**:

- Conversione da **subclass** → **superclass**: **automatica (upcasting)**.
- Conversione da **superclass** → **subclass**: **richiede cast esplicito (downcasting)**.
- Conversione tra tipi **senza relazione**: **non permessa**.

Esempio

```
class Animale {}
class Cane extends Animale {}

Animale a = new Cane(); // upcasting (automatico)
Cane c = (Cane) a; // downcasting (esplicito)

// Animale a2 = new Animale();
// Cane c2 = (Cane) a2; // Compila, ma genera ClassCastException a runtime!
```

✓ Riepilogo

- **Promotion** = automatica, sicura (es. `int` → `long`, `char` → `int`).

- **Casting esplicito** = a rischio perdita di dati (es. `double` → `int`, `int` → `byte`).
 - Con i **reference**:
 - Upcasting = sempre sicuro.
 - Downcasting = possibile ma rischioso (può lanciare `ClassCastException`).
 - **Boolean** = non convertibile con cast.
-

Altri esempi

♦ Metodi in Java

Un **metodo** è un concetto fondamentale della programmazione orientata agli oggetti (OOP). È un **insieme di istruzioni con un nome** che serve a:

- risolvere un problema scomponendolo in sottoproblemi,
- rendere il codice più chiaro e strutturato,
- favorire il riuso del codice.

Sinonimi in altri linguaggi: **funzioni, procedure, subroutines**.

♦ Componenti di un metodo

Una dichiarazione di metodo può contenere fino a **sei parti principali** (alcune opzionali), nell'ordine:

1. **Modificatori** → `public`, `private`, `static`, ecc.
 2. **Tipo restituito** → tipo del valore prodotto (`int`, `String`, `void` se non restituisce nulla).
 3. **Nome del metodo** → segue le regole degli identificatori, ma per convenzione inizia con lettera minuscola e verbo (es. `calcolaArea`).
 4. **Elenco parametri** → tra parentesi `()`, separati da virgole, ognuno con tipo e nome (`int x`, `int y`).
 5. **Elenco di eccezioni** (opzionale) → con `throws`.
 6. **Corpo del metodo** → tra `{ }`, contiene le istruzioni da eseguire.
-

♦ Argomenti attuali e formali

- Quando si **invoca** un metodo, si passano **argomenti attuali**.
- Questi devono corrispondere per numero, ordine e tipo agli **argomenti formali** dichiarati.
- Gli argomenti attuali possono essere variabili o espressioni.

Esempio:

```
public static int somma(int a, int b) {  
    return a + b;  
}  
  
int risultato = somma(5, 7); // argomenti attuali: 5, 7
```

◆ Overloading dei metodi

È possibile definire più metodi con lo **stesso nome** ma firme diverse. La **firma** (signature) = nome del metodo + lista tipi degli argomenti.

```
public static int raddoppia(int x) { return 2 * x; }  
public static String raddoppia(String s) { return s + s; }
```

 **Non è permesso l'overloading solo sul tipo di ritorno.**

◆ Metodi statici (ausiliari)

- Un metodo dichiarato con `static` appartiene alla **classe** e non a una singola istanza.
- Si invoca con `NomeClasse.metodo()`.
- Esempio: `Math.sqrt(16)`.
- In programmi semplici (con `main`) si usano metodi statici **ausiliari** per separare la logica.

Esempio corretto:

```
public class ProvaMetodiStatic {  
    public static void main(String[] args) {  
        metodoUno();  
        metodoDue();  
    }  
  
    public static void metodoUno() {  
        System.out.println("Hello World");  
    }  
  
    public static void metodoDue() {  
        metodoUno();  
        metodoUno();  
    }  
}
```

◆ Quando e perché usare i metodi

1. Per **scomporre un problema complesso** in sottoproblemi gestibili.
 2. Per **riutilizzare** codice già scritto.
 3. Per rendere il codice più leggibile e modulare.
 4. Approccio chiamato anche **programmazione procedurale** o **astrazione funzionale**.
-

♦ Metodi non statici (di istanza)

- Sono operazioni eseguibili su singoli oggetti.
- Si invocano con la sintassi:

```
oggetto.metodo(args);
```

- Ogni parametro ha un tipo e un nome.
- Esempio:

```
String testo = "ciao";  
int lunghezza = testo.length(); // metodo di istanza
```

♦ Metodi predicativi

Un **metodo predicativo** restituisce un **boolean** e può essere usato direttamente in condizioni. Per convenzione, i nomi iniziano con **is** o **has**:

- `isEmpty()`
- `hasNext()`

Esempio:

```
if (lista.isEmpty()) {  
    System.out.println("Lista vuota");  
}
```

Varargs

I varargs (**variable arguments**) sono utili per ogni metodo che abbia a che fare con un numero di argomenti indeterminato.

Nelle API di Java si trova per esempio nel caso del metodo `String.format` che accetta un numero variabile di argomenti.

```
public String formatWithVarArgs(String... values) {
    // ...
}

formatWithVarArgs();

formatWithVarArgs("a", "b", "c", "d");
```

L'utilizzo di varargs è sicuro se e solo se:

- Non memorizziamo nulla nell'array creato implicitamente.
- Non lasciamo che un riferimento all'array generato *sfugga* al metodo

```
public class Fuoristrada extends Automobile {
    private ArrayList<Automobile> updates;
    // Costruttori
    public Fuoristrada () {
        updates = new ArrayList<Automobile>();
    }
    public Fuoristrada (String nome) {
        super(nome);
        updates = new ArrayList<Automobile>();
    }
    public Fuoristrada (String nome, Automobile... caratteristiche) {
        super(nome);
        updates = new ArrayList<Automobile>();
        addUpdates(caratteristiche);
    }
    // Getters e Setters
    public void addUpdate (Automobile update) {
        updates.add(update);
    }
    public void removeUpdate (Automobile update) {
        updates.remove(update);
    }
    public void addUpdates (Automobile... caratteristiche) {
        for (Automobile update : caratteristiche) {
            updates.add(update);
        }
    }
    public void removeUpdates(Automobile... caratteristiche) {
        for (Automobile update : caratteristiche) {
            updates.remove(update);
        }
    }
}
```

✓ Riepilogo

- I **metodi** sono strumenti per organizzare, riutilizzare e rendere leggibile il codice.
 - Possono essere **statici** (di classe) o **di istanza** (su oggetti).
 - Supportano **overloading**, ma non solo per tipo di ritorno.
 - I metodi predicativi (**boolean**) sono fondamentali per condizioni e controlli.
 - I varargs permettono di gestire un numero variabile di argomenti in modo flessibile.
-

codice esempi d'uso

- [raccolta esempi](#)
 - [Esempi sui metodi ausiliari](#)
-

gestire e formattare i numeri

In Java, ci sono diverse modalità per gestire e formattare i numeri, che variano a seconda del tipo di numero (intero, decimale, valuta, percentuale, ecc.). Le principali modalità per lavorare con i numeri sono:

1. Formattazione di numeri interi e decimali

Java offre diversi strumenti per formattare i numeri, come `String.format()`, `DecimalFormat`, e `printf()`.

1.1 `String.format()`

`String.format()` ti consente di formattare i numeri in base a un formato specificato. Ad esempio:

```
int number = 12345;
double price = 1234.56;

String formattedNumber = String.format("%,d", number); // Aggiunge le
virgole come separatori delle migliaia
String formattedPrice = String.format("%.2f", price); // Mostra 2 decimali

System.out.println(formattedNumber); // 12,345
System.out.println(formattedPrice); // 1234.56
```

- `%d`: Formatta i numeri interi.
 - `%f`: Formatta i numeri a virgola mobile (float/double).
 - `%,d`: Aggiunge un separatore di migliaia.
 - `%.2f`: Limita la parte decimale a 2 cifre.
-

1.2 `printf()`

`printf()` funziona come `String.format()`, ma stampa direttamente il risultato sulla console:

```
int number = 12345;
double price = 1234.56;

System.out.printf("%,d%n", number); // 12,345
System.out.printf("%.2f%n", price); // 1234.56
```

1.3 DecimalFormat

`DecimalFormat` è una classe specifica per la formattazione dei numeri decimali. È molto utile se desideri un formato personalizzato (ad esempio, numeri con separatori di migliaia, percentuali, ecc.).

```
import java.text.DecimalFormat;

double number = 12345.6789;
---DecimalFormat df = new DecimalFormat

("#,###.##"); // formato per migliaia e due decimali

System.out.println(df.format(number)); // 12,345.68
```

Alcuni altri pattern di `DecimalFormat`:

- `0`: Forza la visualizzazione di uno zero, se necessario (es. `0.00`).
- `#`: Mostra un numero solo se c'è una cifra.
- `,`: Usato per separare le migliaia.

1.4 Percentuali

Puoi formattare i numeri come percentuali utilizzando `DecimalFormat` o `String.format()`:

```
double percentage = 0.25;
System.out.printf("%.2f%%\n", percentage * 100); // 25.00%
```

Oppure con `DecimalFormat`:

```
DecimalFormat df = new DecimalFormat("#0.00%");
System.out.println(df.format(percentage)); // 25.00%
```

2. Formattazione della valuta

Per la formattazione della valuta, puoi usare `NumberFormat`, che è specifico per i numeri monetari e si adatta alla locale (lingua/paese) configurata.

2.1 `NumberFormat` per la valuta

```
import java.text.NumberFormat;
import java.util.Locale;

double price = 1234.56;

NumberFormat currencyFormat = NumberFormat.getCurrencyInstance(Locale.US);
System.out.println(currencyFormat.format(price)); // $1,234.56
```

Puoi cambiare la `Locale` per adattare la valuta a un altro paese (ad esempio, per l'Italia puoi usare `Locale.ITALY`).

2.2 Formattazione personalizzata con `DecimalFormat`

Anche `DecimalFormat` può essere utilizzato per formattare la valuta:

```
DecimalFormat df = new DecimalFormat("¤#,##0.00"); // Il simbolo della
valuta è aggiunto automaticamente
System.out.println(df.format(price)); // $1,234.56 (se Locale.US è
impostato)
```

3. Utilizzare `BigDecimal` per numeri ad alta precisione

Quando hai bisogno di lavorare con numeri che richiedono un'alta precisione (ad esempio, per applicazioni finanziarie), puoi usare `BigDecimal`, che evita i problemi di arrotondamento tipici di `float` e `double`.

```
import java.math.BigDecimal;
import java.math.RoundingMode;

BigDecimal value = new BigDecimal("1234.5678");
value = value.setScale(2, RoundingMode.HALF_UP); // Arrotonda a due
decimali

System.out.println(value); // 1234.57
```

4. Impostare la lingua/locale per la formattazione

La formattazione dei numeri può variare a seconda della locale, come ad esempio la separazione decimale con il punto o la virgola. Puoi impostare la locale per formattare correttamente i numeri in base alla lingua e al paese dell'utente.

4.1 Impostare la locale per `NumberFormat`

```
import java.text.NumberFormat;
import java.util.Locale;

double number = 12345.6789;

NumberFormat nf = NumberFormat.getInstance(Locale.FRANCE); // Locale per
la Francia
System.out.println(nf.format(number)); // 12 345,6789
```

5. Gestire la precisione nei numeri decimali

Se hai bisogno di controllare la precisione nei calcoli con numeri decimali (come nel caso delle valute o di calcoli scientifici), puoi utilizzare `BigDecimal` o il metodo `setScale()`.

```
BigDecimal result = new BigDecimal("123.456789");
result = result.setScale(2, RoundingMode.HALF_UP); // Arrotonda a 2
decimali
System.out.println(result); // 123.46
```

Riepilogo delle tecniche

- **Concatenazione:** Usa `+` per concatenare numeri e stringhe.
- **`String.format()` e `printf()`:** Formattazione di numeri con specifiche di formato.
- **`DecimalFormat`:** Personalizzazione avanzata di come i numeri vengono formattati (separatori di migliaia, decimali, percentuali, ecc.).
- **`NumberFormat`:** Utilizzato per formattare la valuta e altre forme numeriche locali.
- **`BigDecimal`:** Precisione elevata per operazioni con numeri decimali, utile per evitare imprecisioni nei calcoli finanziari.

Con queste tecniche, puoi gestire e formattare i numeri in Java in modo molto flessibile e adatto alle esigenze della tua applicazione.

La gestione della memoria

La gestione della memoria in Java è affidata al Garbage Collector (GC), un componente del Java Virtual Machine (JVM) responsabile della liberazione della memoria da oggetti non più referenziati. La gestione

automatica della memoria è uno dei principali vantaggi di Java, in quanto allevia lo sviluppatore dal compito di liberare manualmente la memoria allocata.

Ecco alcuni concetti chiave relativi alla gestione della memoria in Java:

1. Heap Memory:

- La memoria heap è la regione di memoria in cui vengono allocate le istanze degli oggetti Java.
 - Gli oggetti in heap persistono per la durata di vita dell'applicazione o fino a quando vengono rilasciati dal Garbage Collector.
-

2. Stack Memory:

- La memoria stack è utilizzata per conservare i dati locali e i riferimenti ai metodi.
 - Le variabili locali e i riferimenti agli oggetti vengono memorizzati nello stack.
 - Lo stack è più veloce rispetto all'heap, ma ha una dimensione limitata.
-

3. Garbage Collector:

- Il Garbage Collector individua e rimuove gli oggetti non più referenziati per liberare memoria.
 - Utilizza vari algoritmi per determinare quali oggetti possono essere considerati "spazzatura".
 - I principali algoritmi GC includono il Garbage-First Collector (G1), il Parallel Collector e il CMS Collector.
-

4. Ciclo di Vita degli Oggetti:

- **Creazione:** Gli oggetti vengono creati utilizzando l'operatore `new` o altri meccanismi come la reflection.
 - **Utilizzo:** Gli oggetti vengono utilizzati nel programma, accedendo ai loro metodi e attributi.
 - **Referenziamento:** Gli oggetti vengono referenziati da variabili, strutture dati o altri oggetti.
 - **De-referenziamento:** Quando un oggetto non è più referenziato, può essere candidato alla raccolta.
 - **Raccolta:** Il Garbage Collector rileva gli oggetti non referenziati e li libera.
-

5. Metodi `finalize()` e `Object.finalize()`:

- La classe `Object` fornisce un metodo `finalize()` chiamato dal GC prima che un oggetto venga rimosso.
 - È sconsigliato affidarsi esclusivamente a `finalize()` per la pulizia delle risorse, e si preferisce l'uso di costruttori e blocchi `try-with-resources`.
-

6. Risorse Non in Heap:

- Le risorse esterne come connessioni di database o file devono essere chiuse esplicitamente dal programmatore per evitare perdite di memoria.

- L'uso di blocchi `try-with-resources` (introdotti in Java 7) semplifica la gestione delle risorse.
-

7. Monitoraggio della Memoria:

- Strumenti come Java VisualVM o strumenti di profilazione possono essere utilizzati per monitorare l'utilizzo della memoria e individuare eventuali perdite.
-

8. Garbage Collection Esplicita:

- Mentre il GC opera automaticamente, è possibile richiedere esplicitamente la raccolta dei rifiuti tramite `System.gc()`. Tuttavia, ciò non garantisce che la raccolta venga effettuata immediatamente.

La gestione automatica della memoria in Java semplifica la vita dello sviluppatore, ma è importante scrivere codice efficiente per minimizzare la pressione sul Garbage Collector e garantire le migliori prestazioni. Evitare la creazione inutile di oggetti, liberare risorse esterne e monitorare l'utilizzo della memoria sono pratiche importanti per un'applicazione Java robusta e performante.

Le classi e gli elementi statici in Java sono memorizzati nello spazio di memoria condiviso chiamato "Metaspace" (prima di Java 8 era chiamato "PermGen" per Permanent Generation).

Classi Statiche

Le classi sono memorizzate in Metaspace. Ogni classe Java, inclusi i suoi metodi e i relativi metadati, viene rappresentata da un oggetto chiamato "java.lang.Class". Questi oggetti `Class` vengono conservati nello spazio Metaspace.

Elementi Statici

Le variabili e i metodi statici appartengono alla classe, non a un'istanza specifica della classe. Pertanto, la memoria per le variabili statiche e i metodi statici è associata alla classe stessa e non agli oggetti istanza. Questa memoria è anch'essa situata nello spazio Metaspace.

Metaspace

Metaspace è una regione di memoria separata dall'heap principale in cui vengono memorizzati i dati relativi alle classi, ai metodi e ad altre informazioni di runtime. A differenza della vecchia PermGen, Metaspace è dinamicamente ridimensionabile e non soggetto a limiti statici come PermGen.

Memoria Heap

Mentre Metaspace è utilizzato per memorizzare informazioni relative alle classi e ai metodi, la memoria heap è responsabile per la memorizzazione delle istanze degli oggetti. Ogni volta che si crea un nuovo oggetto mediante l'operatore `new`, la memoria per quell'oggetto viene allocata nello spazio heap.

In sintesi, le classi e gli elementi statici sono memorizzati nello spazio Metaspace, una regione di memoria separata dallo spazio heap utilizzato per le istanze degli oggetti. L'utilizzo di Metaspace consente una gestione più dinamica e flessibile rispetto alla vecchia PermGen.